Informe Laboratorio 8 - TEL137

Nombres y Apellidos: Alvaro Brandon Pimentel Melgar
Código: 20232283

1. Objetivo del laboratorio

El laboratorio pedía tomar el proyecto base de red social del repositorio gtics-lab8-base y dejarlo funcionando, ya que en el estado en que venía solo tenía a medias la parte de publicar posts. Lo principal a evaluar era el manejo de sesión y seguridad con Spring Security.

Lo que se tenía que entregar, según lo indicado en clase:

- Listar, editar y eliminar posts.
- Enviar solicitudes de amistad a otros usuarios.
- Que cada acción respete el rol del usuario (admin, user o visitante sin sesión).
- Algunas mejoras al código, sin llegar a algo demasiado avanzado.
- Contenerizar la aplicación para que se pueda levantar con Docker.
- Este informe, explicando cómo se usó la IA durante el desarrollo.

Para el desarrollo usé Claude como apoyo, y en este informe explico qué le pedí, qué salió y qué tuve que revisar o corregir.

2. Estado del proyecto base

Errores principales que encontré fueron:

Archivo	Problema	Qué se hizo
Lab5_script.sql	Las líneas --- Roles y --- Users no son comentarios válidos en MySQL (falta el espacio después de --), así que el script fallaba al cargar los datos.	Se cambiaron a -- Roles y -- Users.
PostController	La ruta de crear post estaba como /post/post/create y algunos return usaban vistas que no existían (/post/list).	Se corrigieron las rutas y las redirecciones.
post/lista.html	El <head> estaba mal cerrado, la columna "Autor" mostraba el contenido y el botón de borrar era un <a> vacío. Además, session.rol.equals(...) fallaba cuando no había sesión.	Se rehizo la tabla y se validan los roles sin que falle para visitantes.
post/form.html	Usaba post.idPost y usuario.id_user, campos que no existían en ese formulario.	Se rehizo el formulario con validaciones.
base.html	El navbar llamaba a getNombre() y getApellido(), que no existen en la entidad, y el botón de cerrar sesión no hacía nada.	Se usan firstname y lastname, y el logout es un formulario POST.
UsuarioController	Redirecciones mal escritas ("redirect: /user/list" con espacio) y vistas que no existían (/usuario/form).	Se reescribió el controlador.
Usuario.java	No tenía @GeneratedValue en el id.	Se agregó.
HomeController	No existía el mapeo de /loginForm, que es la página de login configurada en Spring Security.	Se agregó.

Al final de esta parte el proyecto ya compilaba y se podía iniciar sesión y listar posts.

3. Uso de la IA

Usé Claude en modo agente: le subí el .zip del repositorio base y una captura del enunciado del laboratorio en PAIDEIA y la herramienta podía leer los archivos, compilar con Maven, levantar MySQL y probar la aplicación. No le pedí todo de golpe, sino en dos partes.

Primera consulta

Le pasé el zip y la captura del enunciado y le pedí más o menos esto:

"Dame la resolución en zip, se puede usar IA."

Con eso la IA revisó todo el proyecto, encontró los errores de la sección anterior, implementó el listado de posts con permisos por rol, la sesión con los datos del usuario y agregó el Dockerfile y el docker-compose.

Segunda consulta

Después de la clase, el JP dio más detalles de lo que se iba a evaluar, así que le pasé a la IA lo que había anotado:

"Completar las funcionalidades: listar, eliminar, editar posts. Enviar solicitud de amistad a otros usuarios. Entregar un informe detallado de uso de IA, implementar mejoras al código pero que sean considerables buenas o normales para el curso, nada exagerado. Contenerizar la aplicación (capaz de subirse con Docker). Las acciones deben ser acorde al rol. El código no debe ser nada complejo, el tema a evaluar es Sesión y Seguridad."

Ahí la IA agregó la parte de solicitudes de amistad (tabla nueva, entidad, repositorio, controlador y vista) y volvió a probar todo.

Cómo trabajé con la IA

- Le di el contexto completo (enunciado, repositorio y lo que dijo el JP) en vez de pedir cosas sueltas, porque así el resultado salía más coherente con lo que se pedía.
- Le puse restricciones claras: código sencillo y enfocado en sesión y seguridad. Sin eso tendía a proponer cosas más elaboradas de lo necesario para el curso.
- Le pedí que probara la aplicación antes de entregarme el zip, no solo que compilara.
- Revisé el código que generó para poder explicarlo, sobre todo `WebSecurityConfig`, `PostController` y `AmistadController`, que es donde está la lógica de permisos.

4. Desarrollo

4.1 Sesión

Cuando el usuario inicia sesión, Spring Security lo manda a `/redirectByRol`. Ahí se busca el usuario por su correo y se guardan dos atributos en la sesión: `usuario` y `rol`. Con eso el navbar muestra el nombre y las opciones de cada rol, y los controladores saben quién está haciendo cada acción.

La sesión se guarda en la base de datos con Spring Session JDBC (tablas `SPRING_SESSION`), así que no se pierde si se reinicia la aplicación. Tiene un tiempo de expiración de 30 minutos. Al cerrar sesión se invalida la sesión y se borran las cookies. Si el usuario edita su perfil, también se actualiza el objeto guardado en la sesión para que el navbar muestre el nombre nuevo.

```java
@GetMapping("/redirectByRol")
public String redirectByRole(Authentication auth, HttpSession httpSession) {
    String rol = "";
    for (GrantedAuthority role : auth.getAuthorities()) {
        rol = role.getAuthority();
        break;
    }
    Usuario usuario = usuarioRepository.findByEmail(auth.getName());
    httpSession.setAttribute( s: "usuario", usuario);
    httpSession.setAttribute( s: "rol", rol);

    if (rol.equals("admin")) {
        return "redirect:/post";
    } else if (rol.equals("user")) {
        return "redirect:/post";
    }
    return "redirect:/";
}
```

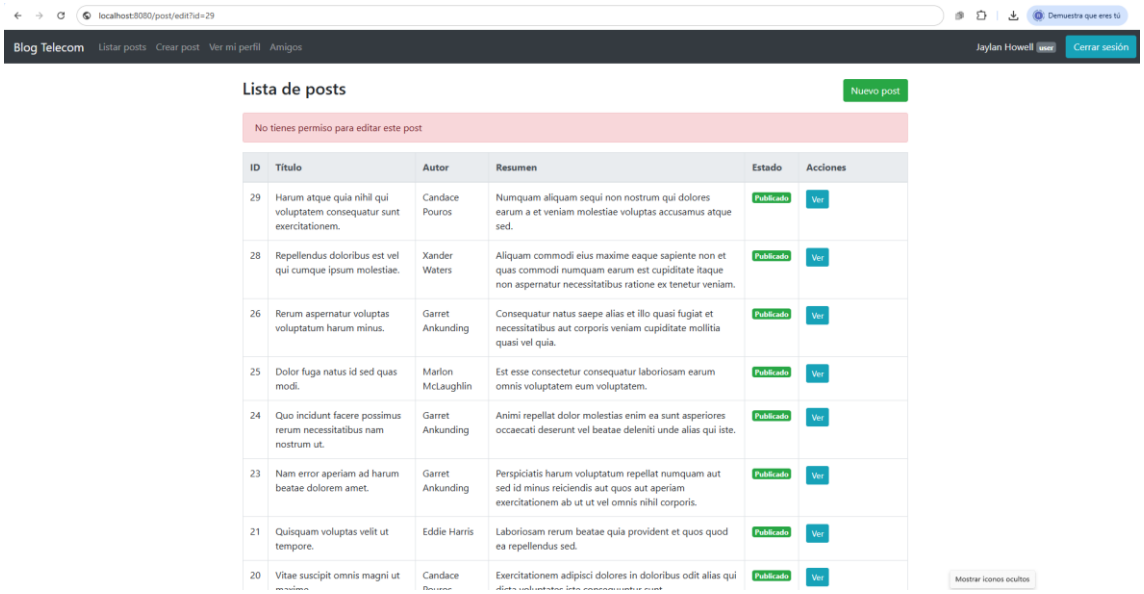
4.2 Seguridad

En `WebSecurityConfig` se configuró:

- Login propio en `/loginForm`, que valida contra la tabla `user` con contraseñas en BCrypt. Los usuarios con `active = 0` no pueden entrar.
- Logout por POST, porque con CSRF activado un logout por GET no funciona.
- Protección CSRF en todos los formularios (Thymeleaf agrega el token solo con `th:action`). Por eso borrar un post o aceptar una solicitud ahora es un formulario POST y no un enlace.
- Protección contra fijación de sesión (`migrateSession`), para que al iniciar sesión se genere un id de sesión nuevo.
- Si alguien entra a una página sin permiso, se le redirige al listado con un mensaje de "no tienes permisos" en vez de la página de error 403.

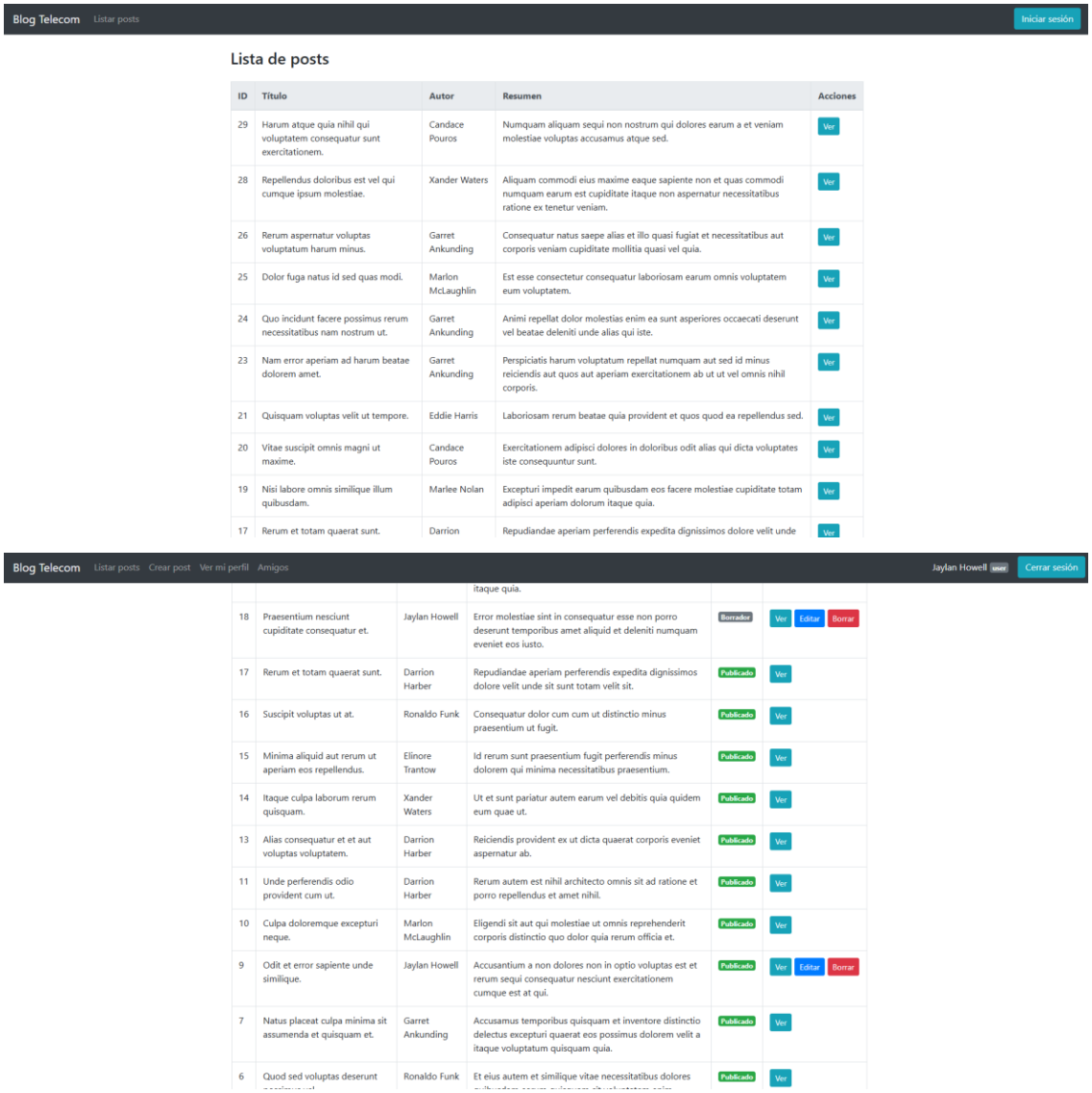
```java
http.authorizeRequests()
        .antMatchers( ...antPatterns: "/user/list", "/user/editar", "/user/actualizar", "/user/estado").hasAuthority("admin")
        .antMatchers( ...antPatterns: "/user", "/user/guardar").hasAnyAuthority( ...authorities: "admin", "user")
        .antMatchers( ...antPatterns: "/post/create", "/post/edit", "/post/guardar", "/post/borrar").hasAnyAuthority( ...authorities: "admin", "user")
        .antMatchers( ...antPatterns: "/amigos", "/amigos/**").hasAnyAuthority( ...authorities: "admin", "user")
        .antMatchers( ...antPatterns: "/redirectByRol").authenticated()
        .anyRequest().permitAll();
```

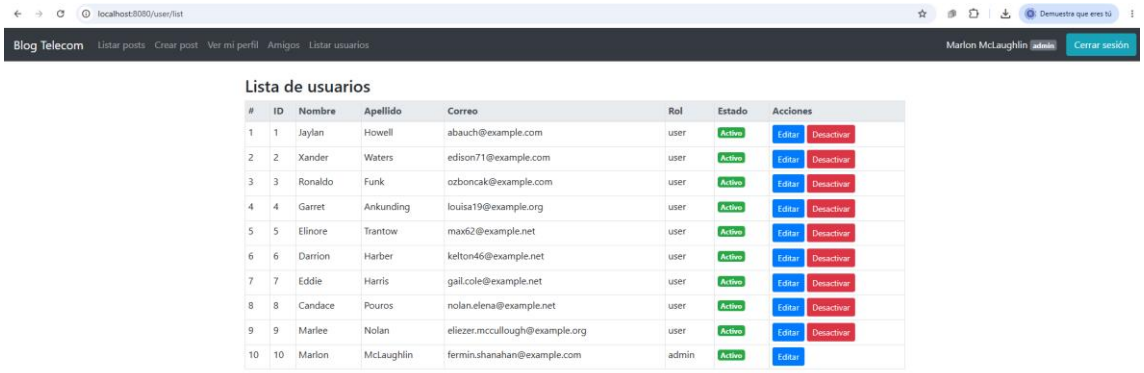
Además de las reglas por URL, los controladores vuelven a verificar que el usuario sea dueño del post antes de editarlo o borrarlo. Esto es importante porque las reglas por URL solo revisan el rol, y sin esta validación un user podría cambiar el id en la URL y editar el post de otra persona.



4.3 Permisos por rol

Acción	Visitante	user	admin
Ver posts publicados	Sí	Sí	Sí
Ver borradores	No	Solo los suyos	Todos
Crear post	No	Sí	Sí
Editar / borrar post	No	Solo los suyos	Cualquiera
Ver y editar su perfil	No	Sí	Sí
Solicitudes de amistad	No	Sí	Sí
Listar y editar usuarios	No	No	Sí
Activar / desactivar usuarios	No	No	Sí (menos a sí mismo)





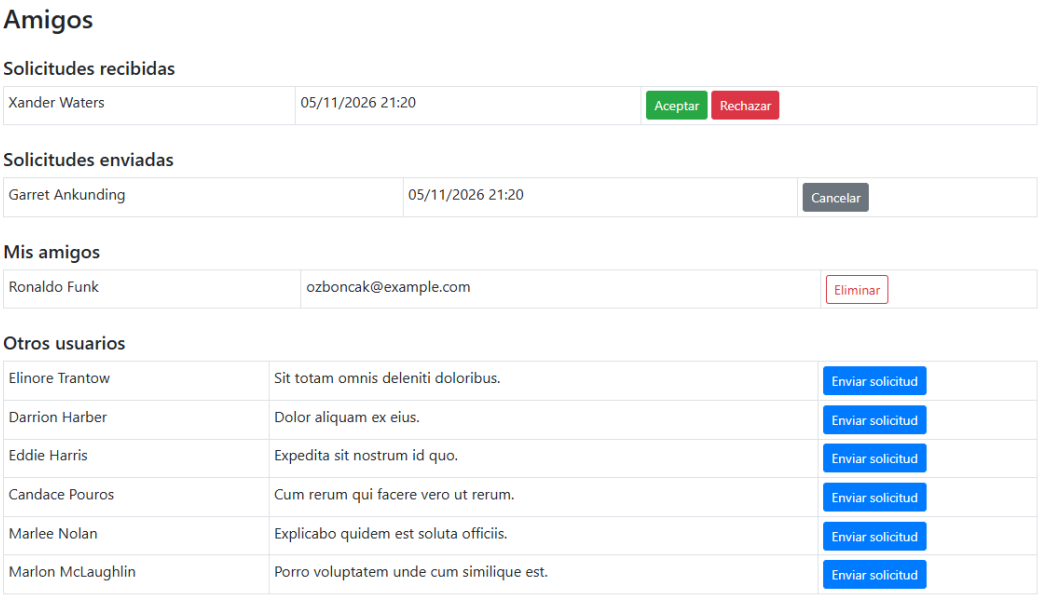
4.4 Posts

- Listar: el listado cambia según quién lo ve. El visitante solo ve los publicados, el user ve los publicados y sus borradores, y el admin ve todos. También se agregó una vista de detalle (`/post/ver`).
- Crear y editar: se usa el mismo formulario. Al crear, el autor se toma de la sesión y no del formulario, para que nadie pueda publicar a nombre de otro. Al editar se mantiene el autor original.
- Eliminar: es un POST con confirmación en el navegador. Solo lo puede hacer el dueño o el admin.

4.5 Solicitudes de amistad

Se agregó la tabla `friend_request` al script SQL, con `sender_id`, `receiver_id`, `status` y `created_at`. El estado solo puede ser `pendiente` o `aceptada`; si una solicitud se rechaza o se cancela, simplemente se borra, para que después se pueda volver a enviar.

En la página `/amigos` cada usuario ve cuatro listas: solicitudes recibidas (aceptar o rechazar), solicitudes enviadas (cancelar), sus amigos (eliminar amistad) y los demás usuarios (enviar solicitud).



Las validaciones que hace el `AmistadController` son:

- No se puede enviar una solicitud a uno mismo ni a un usuario inactivo.
- No se puede enviar si ya existe una solicitud o amistad entre los dos, en cualquier dirección.
- Solo quien recibió la solicitud puede aceptarla o rechazarla.
- Solo quien la envió puede cancelarla.
- Solo los dos involucrados pueden eliminar una amistad.

El usuario que hace cada acción siempre se toma de la sesión, nunca de un parámetro del formulario.

5. Mejoras al código

Como el JP pidió mejoras que no fueran exageradas, me limité a cosas que se han visto en el curso:

1. Validaciones con Bean Validation (`@NotBlank`, `@Size`) en posts y perfil, con los errores mostrados debajo de cada campo.
2. Mensajes flash ("Post creado exitosamente", "No tienes permiso...") con `RedirectAttributes`, en lugar de redirigir sin avisar nada.
3. Acciones que modifican datos por POST (borrar, aceptar, rechazar, logout) para que funcionen con CSRF.
4. Estado publicado/borrador en los posts, usando el campo `published` que ya existía en la tabla pero no se usaba.
5. Perfil propio: cada usuario puede editar su nombre, apellido, introducción y perfil, pero no su correo ni su rol.
6. Activar/desactivar usuarios desde el admin en vez de borrarlos, porque borrar un usuario con posts daría error por la llave foránea.
7. Nombres de campos más claros en las entidades (`idPost`, `idUser`, `rol` en vez de `id_Post` o `id_role`) usando `@Column` para mantener las columnas de la base de datos.
8. Configuración por variables de entorno (`DB_URL`, `DB_USER`, `DB_PASSWORD`) con valores por defecto para local, así el mismo código funciona en local y en Docker.

6. Contenerización

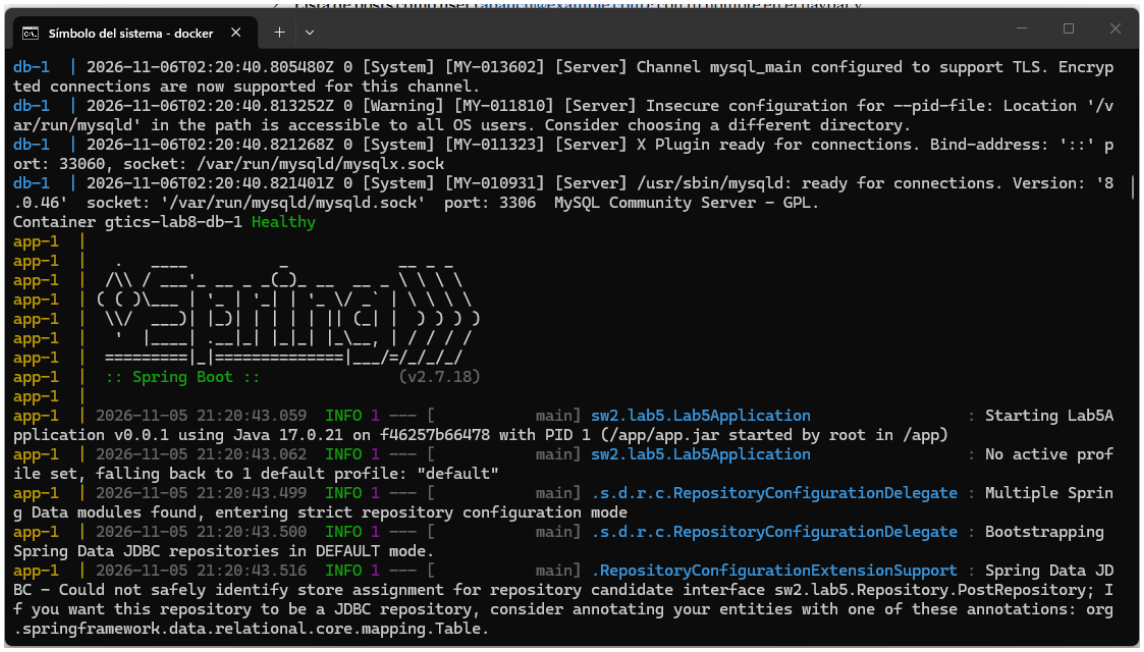
Se agregaron tres archivos:

- Dockerfile en dos etapas: la primera usa la imagen de Maven para compilar el .jar y la segunda solo copia el .jar a una imagen con Java 17 (JRE), para que la imagen final sea más liviana.
- docker-compose.yml con dos servicios: `db` (MySQL 8.0, que carga `Lab5_script.sql` al crearse) y `app` (la aplicación). La app espera a que MySQL esté listo gracias al `healthcheck` y a `depends_on`.
- .dockerignore para no copiar la carpeta `target` ni archivos del IDE.

Para levantar todo basta con:

```
docker compose up --build
```

y entrar a http://localhost:8080. MySQL queda expuesto en el puerto 3307 para no chocar con un MySQL que ya esté instalado en la máquina. Si se modifica el script SQL, hay que correr `docker compose down -v` antes, porque el script solo se ejecuta la primera vez que se crea el volumen.



7. Pruebas

La IA levantó la aplicación con MySQL, cargó el script y probó los flujos con un script en Python que inicia sesión con distintos usuarios y revisa las respuestas. Se usaron estas cuentas de prueba:

Usuario	Rol
fermin.shanahan@example.com	admin
abauch@example.com	user
max62@example.net	user

Se probaron 38 casos: 37 pasaron a la primera y el otro fallaba por un error en la propia prueba (se explica en la sección 8). Los más importantes para el tema de sesión y seguridad fueron:

- Un visitante que entra a `/post/create`, `/user/list` o `/amigos` es mandado al login.
- Un visitante no puede ver un borrador aunque ponga su id en la URL.
- Un user no puede entrar a la lista de usuarios.
- Un user no puede editar, borrar ni sobrescribir (mandando el id por POST) un post de otro usuario.
- Al editar el perfil enviando el id de otro usuario, solo se modifica el perfil propio.

- Después del logout ya no se puede entrar a páginas protegidas.
- Un usuario desactivado por el admin no puede iniciar sesión.
- Un user no puede aceptar una solicitud que no le enviaron ni eliminar una amistad ajena.
- No se puede enviar una solicitud a uno mismo ni duplicarla en sentido inverso.
- Una petición POST sin token CSRF es rechazada.
- La sesión se mantiene en la base de datos después de reiniciar la aplicación.